

Пловдивски университет „Паисий Хилендарски“

Дисциплина: Управление на софтуерни проекти

КУРСОВ ПРОЕКТ

**Система за управление на дипломни
работи (СУДР)**

Изготвил: Галин Петров
Факултетен номер: СИ 2301322006

Пловдив, 2025 г.

1. Описание на проблема и изисквания на клиента

Проблем

В повечето висши училища процесът по дипломни работи се управлява ръчно и разпокъсано. Комуникацията между студент и научен ръководител върви по имейл или лично, темите се избират на хартия или в споделени таблици, няма централно място, където да се проследява напредъкът. Това води до пропуснати срокове, изгубени версии на документи, неясноти кой какво е одобрил и затруднено насрочване на защитите.

Заинтересовани страни на високо ниво

Студенти, научни ръководители, катедри и деканати, администрация (учебен отдел), комисии по защита.

Минимални изисквания (задължителни)

- Регистрация и вход с университетски профил (студент, преподавател, администратор).
- Каталог с предложени теми за дипломни работи от преподавателите.
- Подаване на заявка от студент за тема и одобрение от преподавател.
- Проследяване на статус на дипломната работа (избрана тема, в разработка, предадена, допусната до защита, защитена).
- Качване на версии на документи с история на версиите.
- Коментари и обратна връзка от научния ръководител по качен документ.
- Известия по имейл при важни събития (одобрена тема, нов коментар, приближаващ срок).

Желани изисквания (ако има време/ресурс)

- Насрочване на защиты и генериране на график.
- Проверка за плагиатство (интеграция с външна услуга).
- Табло (dashboard) със статистика за катедрата.
- Мобилна версия / адаптивен интерфейс.
- Експорт на протоколи и документи в PDF.

Ограничения

- Да работи в уеб браузър.
- Да спазва изискванията за защита на лични данни (GDPR).
- Да се интегрира с университетската система за автентикация, ако има такава.

2. Общ преглед на проекта

Кратко описание на възможността

Разработване на уеб базирана система, която централизира целия жизнен цикъл на дипломната работа - от избора на тема до защитата. Системата заменя имейл комуникацията и ръчното проследяване с единна платформа.

Цели и очаквани ползи

- Намаляване на административните усилия и пропуснатите срокове.
- Прозрачност на процеса за всички страни.

- Единно и проследимо място за версии и обратна връзка.
- По-бързо и организирано насрочване на защиты.

Първоначален списък със заинтересовани страни

Студенти, научни ръководители, ръководител на катедра, учебен отдел/администрация, ИТ отдел на университета, комисия по защита, разработчик (аз).

Екип и организационна структура

Проектът се изпълнява от един човек, който поема няколко роли от Scrum и разработката:

- Product Owner - определя приоритетите и управлява product backlog.
 - Scrum Master (олекотена роля) - следи процеса и премахва пречки.
 - Developer - проектира, разработва и тества системата.
- „Клиент“ в проекта е условно представен от научен ръководител/катедра, който дава изискванията и приема резултатите.

Обхват на проекта (какво влиза)

- Анализ и документиране на изискванията.
- Проектиране на базата данни и архитектурата.
- Разработка на минимално работещ продукт (MVP) с минималните изисквания.
- Тестване и демонстрация.
- Изготвяне на проектната документация и презентация.

Извън обхвата (какво не влиза)

- Реална интеграция с продукционната система на университета.
- Пълна услуга за плагиатство (само като желана функция/макет).
- Поддръжка и експлоатация след предаването.

3. Обоснован избор на методология

Сравняват се две методологии в контекста на този проект: планова (Waterfall) и гъвкава (Scrum / Agile).

Waterfall (планова)

Силни страни: ясен, последователен план; подходяща при стабилни и известни отнапред изисквания; лесна за документиране и отчитане; предвидими срокове и бюджет.

Слаби страни: трудно поема промени в изискванията; резултат се вижда чак в края; висок риск, ако нещо в началните фази е сгрешено; малко обратна връзка по пътя.

Scrum (гъвкава)

Силни страни: работи на кратки итерации (спринтове) с работещ резултат на всеки спринт; лесно поема промени в изискванията; ранна и честа обратна връзка; приоритизиране на най-ценните функции първо; подходяща когато изискванията се избистрят в процеса.

Слаби страни: изисква дисциплина и редовни срещи; по-малко предвидим обхват и срок; документацията може да изостане; при един човек някои Scrum церемонии стават формални.

Избор и обосновка

За проекта се избира **гъвкава методология (Scrum)**. Причините:

- Изискванията за дипломната система не са напълно фиксирани в началото и се избистрят с обратна връзка от научния ръководител.
- Работата на спринтове позволява ранно да имаме работещ MVP, а по-рисковите функции да се оставят за по-късно.
- Приоритизацията чрез product backlog гарантира, че минималните изисквания се правят преди желаните.
- Кратките итерации намаляват риска - проблемите се откриват рано, а не в края. Scrum се прилага в олекотен вид заради работата от един човек: кратки спринтове, личен backlog, преглед и ретроспекция в по-малък формат.

4. Регистър на рисковете

Име на риска	Описание на риска	Стратегии за смекчаване/управление	Вероятност	Въздействие
Неясни изисквания	Изискванията на клиента се променят или не са добре дефинирани в началото	Чести прегледи с научния ръководител; приоритизиран backlog; работа на спринтове с обратна връзка	Висока	Високо
Недостиг на време	Един човек не успява да покрие целия обхват в срок	Фокус върху MVP и минималните изисквания; желаните функции като опционални; реалистична оценка в story points	Висока	Високо
Технически трудности	Непознати технологии или интеграции (автентикация, имейл)	Ранно прототипиране на рисковите части; изучаване на документация; резервни по-прости решения	Средна	Средно
Загуба на данни/код	Повреда на устройство или изгубени версии на документи	Контрол на версиите (Git); редовни резервни копия в облак	Ниска	Високо
Защита на лични данни	Неправилно съхранение на лични данни на студенти	Съобразяване с GDPR; криптиране на паролите; минимизиране на събираните данни	Средна	Високо
Прекомерен обхват (scope creep)	Добавяне на все нови функции извън планираното	Строго управление на backlog от Product Owner; ясно разделение минимални/желани изисквания	Средна	Средно

5. Структура на разбивка на работата (WBS)

1. Управление на проекта

- 1.1 Планиране и дефиниране на обхват
- 1.2 Управление на рискове
- 1.3 Проследяване на спринтове и отчитане

2. Анализ и изисквания

- 2.1 Събиране на изисквания от заинтересовани страни
- 2.2 Изготвяне на потребителски истории
- 2.3 Приоритизиране на product backlog

3. Проектиране (Design)

- 3.1 Проектиране на архитектурата
- 3.2 Проектиране на база данни
- 3.3 Проектиране на потребителски интерфейс (макети)

4. Разработка (Development)

- 4.1 Автентикация и роли (студент, преподавател, админ)
- 4.2 Модул „Каталог с теми“
- 4.3 Модул „Заявка и одобрение на тема“
- 4.4 Модул „Проследяване на статус“
- 4.5 Модул „Качване на документи и версии“
- 4.6 Модул „Коментари и обратна връзка“
- 4.7 Модул „Известия по имейл“

5. Тестване

- 5.1 Функционално тестване на всеки модул
- 5.2 Интеграционно тестване
- 5.3 Поправка на дефекти

6. Внедряване и демонстрация

- 6.1 Подготовка на тестова среда
- 6.2 Демонстрация пред клиента

7. Документация и презентация

- 7.1 Проектна документация
- 7.2 Подготовка на финална презентация

6. План за изпълнение (гъвкав)

Основни потребителски истории (Product Backlog)

Оценени в story points по скалата на Фибоначи (1, 2, 3, 5, 8, 13), където точките измерват усилие/сложност, не часове.

ID	Потребителска история	Точки
US1	Като потребител искам да се регистрирам и да вляза с профил, за да имам защитен достъп според ролята ми	5
US2	Като преподавател искам да публикувам теми за дипломни работи, за да ги виждат студентите	3
US3	Като студент искам да разгледам каталога с теми, за да избира подходяща тема	2
US4	Като студент искам да подам заявка за тема, а преподавателят да я одобри или откаже	5
US5	Като потребител искам да виждам статуса на дипломната работа, за да знам на какъв етап е	3
US6	Като студент искам да качвам версии на документа, за да ги преглежда ръководителят	8
US7	Като ръководител искам да оставям коментари по качен документ, за да давам обратна връзка	5
US8	Като потребител искам да получавам имейл известия при важни събития, за да не пропускам срокове	5
US9	Като администратор искам табло със статистика, за да следя процеса в катедрата (желана)	8
US10	Като администратор искам да насрочвам защиты и график (желана)	13

Общо усилие: приоритетните истории US1-US8 (минимален продукт) = 36 точки; желаните US9-US10 = 21 точки.

Основен преглед на техническия процес (срещи, време, ритъм)

- **Дължина на спринт:** 2 седмици.
- **Брой спринтове:** 4 спринта за MVP (около 8 седмици) + 1 буферен спринт за желани функции и полиране.
- **Планиране на спринт (Sprint Planning):** избор на истории от backlog според капацитета (около 8-10 точки на спринт).
- **Дневен преглед (личен Daily):** какво е готово, какво следва, какви пречки.
- **Преглед на спринт (Sprint Review):** демонстрация на готовото пред научния ръководител и обратна връзка.
- **Ретроспекция (Retrospective):** кратък личен преглед какво да се подобри.
- **Инструменти:** Git за версии на кода, дъска със задачи (Trello/GitHub Projects) за backlog.

Примерно разпределение по спринтове

- Спринт 1: US1 (5) + US3 (2) + част от проектиране = основа и вход.
- Спринт 2: US2 (3) + US4 (5) = теми и заявки.
- Спринт 3: US5 (3) + US6 (8) = статус и документи.

- Спринт 4: US7 (5) + US8 (5) = обратна връзка и известия.
- Спринт 5 (буфер): US9/US10 частично + тестване и документация.

Критерии за приемане (примери по история)

- **US1 (вход и роли):** потребител може да се регистрира и влезе; грешна парола дава съобщение за грешка; след вход вижда интерфейс според ролята си; паролите се съхраняват криптирани.
- **US2 (публикуване на теми):** преподавател може да създаде тема със заглавие и описание; темата се появява в каталога; само авторът може да я редактира.
- **US3 (каталог):** студент вижда списък с всички активни теми; може да търси/филтрира по ключова дума.
- **US4 (заявка и одобрение):** студент подава заявка; преподавателят може да я одобри или откаже; при одобрение темата се свързва със студента и излиза от свободния каталог.
- **US5 (статус):** статусът се вижда ясно и се обновява при промяна на етапа; само оторизирани роли го променят.
- **US6 (качване на версии):** студент качва файл; системата пази всички предишни версии с дата; ръководителят може да сваля всяка версия.
- **US7 (коментари):** ръководителят оставя коментар по версия; студентът го вижда; има дата и автор.
- **US8 (известия):** при одобрена тема, нов коментар или приближаващ срок системата изпраща имейл до съответния потребител.

7. Анализ на заинтересованите страни и комуникационен план

Основен регистър на заинтересованите страни

Заинтересована страна / длъжност	Влияние (1-5)	Начин на влияние	Интерес (1-5)
Научен ръководител (клиент)	5	Дава изисквания, приема резултати, обратна връзка	5
Студенти (крайни потребители)	3	Използват системата, дават обратна връзка за ползваемост	5
Преподаватели (крайни потребители)	4	Публикуват теми, одобряват, оценяват функции	4
Ръководител на катедра	4	Одобрява процеса, политики, приоритети	3
Учебен отдел / администрация	3	Административни изисквания, насрочване на защиты	3
ИТ отдел на университета	3	Изисквания за сигурност и интеграция	2
Разработчик (аз)	5	Проектира и изпълнява проекта	5

Комуникационен план

Категория информация	С кого	Начин и тон	Отговорник	Кога / колко често
Състояние на проекта	Научен ръководител	Имейл, официален	Разработчик	В края на всеки спринт (на 2 седмици)
Постигнати резултати (демо)	Научен ръководител, преподаватели	Среща, официален	Разработчик	Sprint Review, на 2 седмици
Планиране и приоритети	Научен ръководител	Среща/имейл, официален	Разработчик (PO)	В началото на всеки спринт
Рискове и проблеми	Научен ръководител	Имейл/телефон, официален	Разработчик	При възникване + месечен преглед
Промени на обхвата	Научен ръководител, катедра	Имейл, официален	Разработчик	При възникване
Обратна връзка от потребители	Студенти, преподаватели	Анкета/разговор, неофициален	Разработчик	След всяко демо
Напредък по задачи	Лично (дъска)	Trello/GitHub, неофициален	Разработчик	Ежедневно